DESIGN CONCEPTS

COMP6100001 - SOFTWARE ENGINEERING
SESSION 05

UNIVERSITAS BINA NUSANTARA

SUBJECT MATTER EXPERT
Maulin Nasari, S.T., M.Kom.

SESSION 05 – DESIGN CONCEPTS

LEARNING OBJECTIVES

At the end of this session, students will be expected to be able to:

- Explain the role of design in SE
- Describe design models and their evolution
- Apply core design concepts (abstraction, modularity, etc.)
- Analyze designs using cohesion & coupling
- Differentiate analysis vs design classes
- Interpret design modeling principles (traceability, simplicity, agility)

SESSION 05 – DESIGN CONCEPTS

CONTENT OUTLINE

- Design Within the Context of Software Engineering
- The Design Process
- Design Concepts
- The Design Model

DESIGN WITHIN THE CONTEXT OF SOFTWARE ENGINEERING

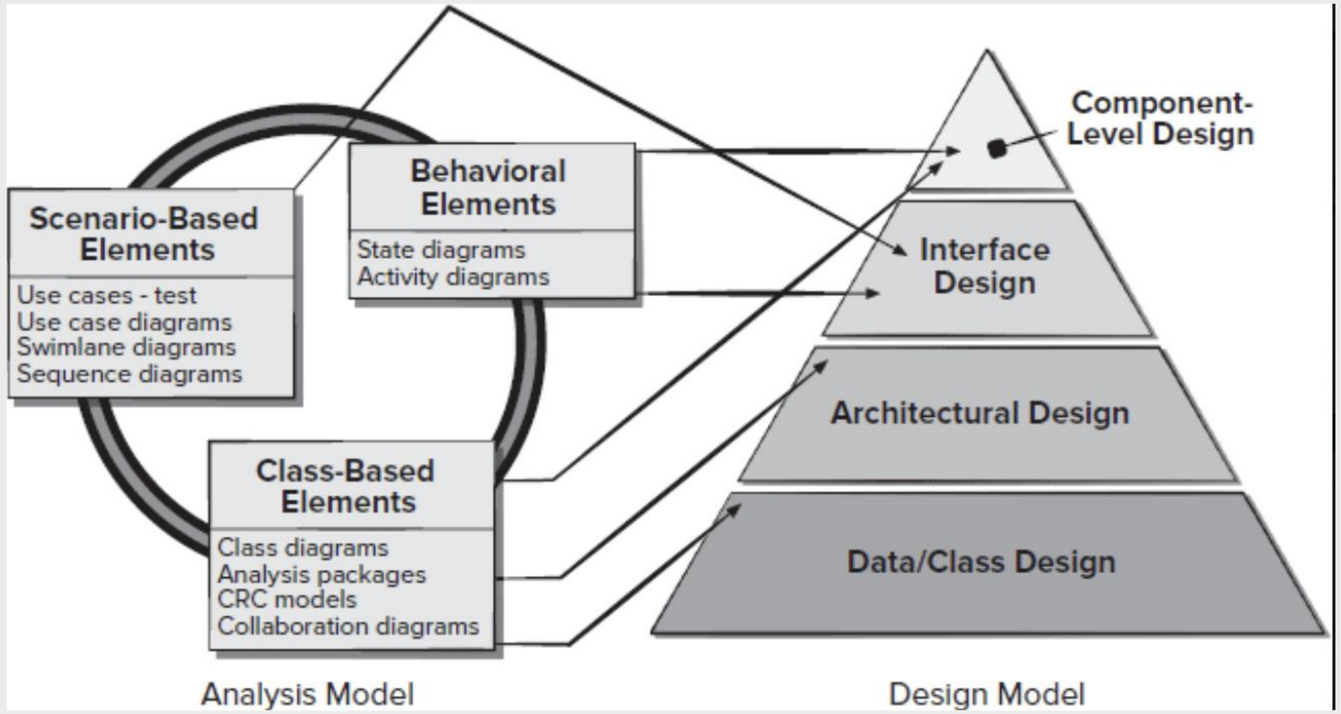
- Software design is the **technical core** of software engineering
- Begins **after requirements analysis and modeling**
- Final activity in the **modeling phase**
- Sets the foundation for:
  - Construction (coding & testing)
  - Maintenance

Design translates what is needed into how it will be built.

DESIGN WITHIN THE CONTEXT OF SOFTWARE ENGINEERING

- Requirements models feed four design models:
  - Data / Class Design
  - Architectural Design
  - Interface Design
  - Component-Level Design
- Design decisions directly affect:
  - Software quality
  - Stability
  - Maintainability

Design is where software quality is created. Poor design leads to fragile, hard-to-maintain systems.



THE DESIGN PROCESS

WHAT DEFINES A GOOD SOFTWARE DESIGN?

- A good design **implements all explicit and implicit requirements**
- A good design is **clear and understandable** for developers, testers, and maintainers
- A good design provides a **complete view** of:
  - Data
  - Function
  - Behavior

Design quality is not about elegance alone. It’s about correctness, clarity, and completeness.

THE DESIGN PROCESS

SOFTWARE DESIGN QUALITY GUIDELINES

- **Architecture**
  - Uses recognizable styles or patterns
  - Supports incremental and testable implementation
- **Modularity**
  - Logically partitioned components
  - Independent functional units
- **Clarity & Structure**
  - Separate views for data, architecture, interfaces, and components
  - Clear and simple interfaces
- **Process & Communication**
  - Derived from requirements analysis
  - Represented using clear and consistent notation

Quality design is intentional, achieved through principles, methods, and review.

DESIGN CONCEPTS

FUNDAMENTAL DESIGN CONCEPTS

- Design concepts provide a foundation for all software design
- Help designers:
  - Partition systems into components
  - Separate what from how
  - Evaluate technical design quality
- Focus on building the right software, not just working software

Design concepts guide thinking before tools or code are involved.

DESIGN CONCEPTS

ABSTRACTION

- Abstraction describes a solution at **multiple levels**
  - High-level: problem domain (e.g., user story)
  - Mid-level: system behavior (e.g., use case)
  - Low-level: implementable detail (e.g., pseudocode)
- Two types of abstraction:
  - **Procedural abstraction:** what a function does
  - **Data abstraction:** what data represents

DESIGN CONCEPTS

ARCHITECTURE

- Defines the **overall structure** of the system
- Describes:
  - Major components
  - Interactions between components
  - Organization of data
- Addresses both:
  - **Structural properties** (components & connections)
  - **Extra-functional properties** (performance, security, reliability)

Architecture provides the framework for all detailed design decisions.

DESIGN CONCEPTS

PATTERNS

- A design pattern is a **proven solution** to a recurring design problem
- Captures:
  - Context
  - Problem
  - Solution
  - Constraints (“forces”)
- Helps designers:
  - Reuse design knowledge
  - Reduce design time
  - Improve consistency

Patterns encode experience — not code.

DESIGN CONCEPTS

SEPARATION OF CONCERNS

- Complex problems are easier when **divided into smaller concerns**
- Each concern:
  - Can be designed independently
  - Reduces overall complexity
- Leads to:
  - Modularity
  - Functional independence
  - Refinement

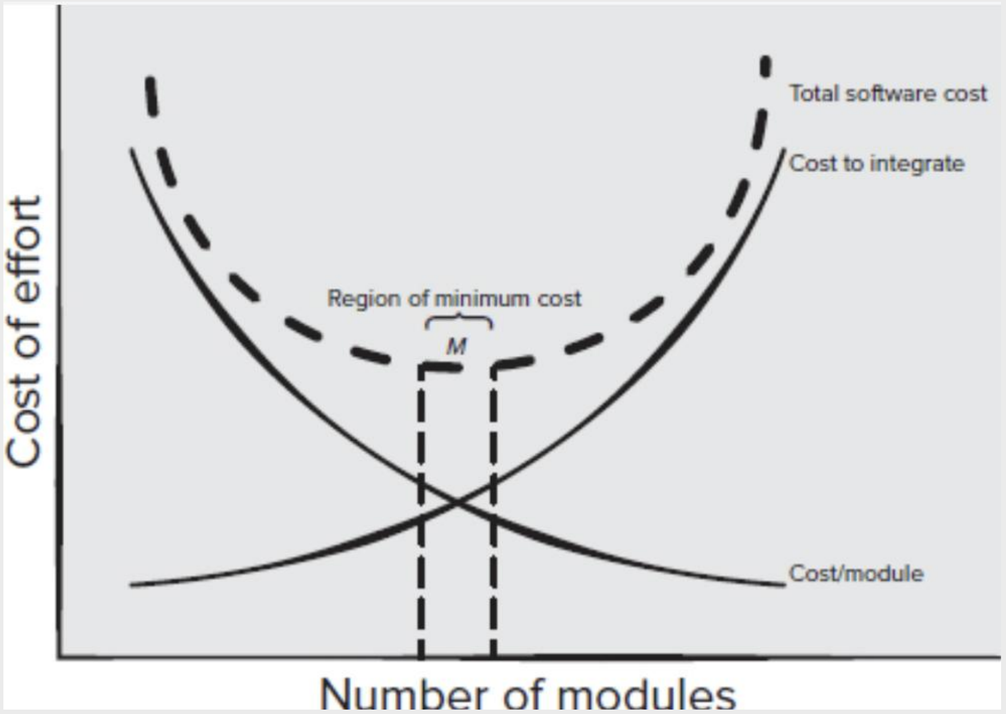
Divide-and-conquer is the basis of scalable software design.

DESIGN CONCEPTS

MODULARITY

- Modularity divides software into separate, manageable modules
- Makes software:
  - Easier to understand
  - Easier to build and test
  - Easier to change and maintain
- **Too few modules** → complex, hard to understand (monolithic)
- **Too many modules** → high integration cost
- There exists an optimal range of modularity (M) where total cost is minimized

Modularity reduces complexity — but excessive modularization increases integration cost.



DESIGN CONCEPTS

INFORMATION HIDING

Key Idea
- Modules should **hide internal data and algorithms**
- Only expose what other modules **need to know**

What is Hidden?
- Internal data structures
- Algorithm details
- Local design decisions

Why It Matters
- Reduces ripple effects when changes occur
- Prevents accidental misuse of internal logic
- Makes maintenance and evolution safer

Change inside a module should not force changes outside it.

DESIGN CONCEPTS

FUNCTIONAL INDEPENDENCE (COHESION & COUPLING)

Key Idea
- Each module should do **one focused job**
- Interaction with other modules should be **minimal and simple**

Two Quality Indicators
- **High Cohesion** → module does one well-defined task
- **Low Coupling** → minimal dependency on other modules

Why It Matters
- Easier development and team collaboration
- Easier testing and debugging
- Less error propagation
- Better reusability

Modules that do many unrelated things (“schizophrenic modules”)

DESIGN CONCEPTS

STEPWISE REFINEMENT

Key Idea

- Design software **top-down**, from abstract to detailed
- Gradually refine until implementable logic is reached

How It Works

1. Start with a high-level description of function
2. Break it into smaller steps
3. Repeat until code-level detail is clear

Relationship to Abstraction

- **Abstraction** hides detail
- **Refinement** reveals detail progressively

Why It Matters

- Controls complexity
- Improves clarity of design decisions
- Produces a traceable design flow

DESIGN CONCEPTS

REFACTORING

Key Idea
Improve internal design **without changing behavior**

What Refactoring Fixes
- Low cohesion
- Redundant or unused elements
- Poor structure or inefficient logic

Why It Matters
- Keeps design clean as software evolves
- Reduces technical debt
- Makes future changes easier and safer

Important Note
- Refactoring must be supported by testing
- Behavior must remain unchanged

Refactoring improves design quality, not functionality.

DESIGN CONCEPTS

DESIGN CLASSES

From Analysis to Design

- **Analysis classes** → describe *problem-domain concepts* (what users see)
- **Design classes** → refine analysis classes with *technical detail* for implementation
- As architecture emerges, abstraction is **reduced** and classes become code-ready

Purpose of Design Classes

- Define attributes + methods needed for implementation
- Provide a blueprint for developers
- Support the overall software architecture

DESIGN CONCEPTS

DESIGN CLASSES

Characteristics of a Well-Formed Design Class

1. Complete & Sufficient
- Contains *all* attributes and methods reasonably expected
- No missing responsibilities
- No unnecessary functionality

2. Primitive
- Each method does **one clear service**
- No duplicate ways to perform the same action

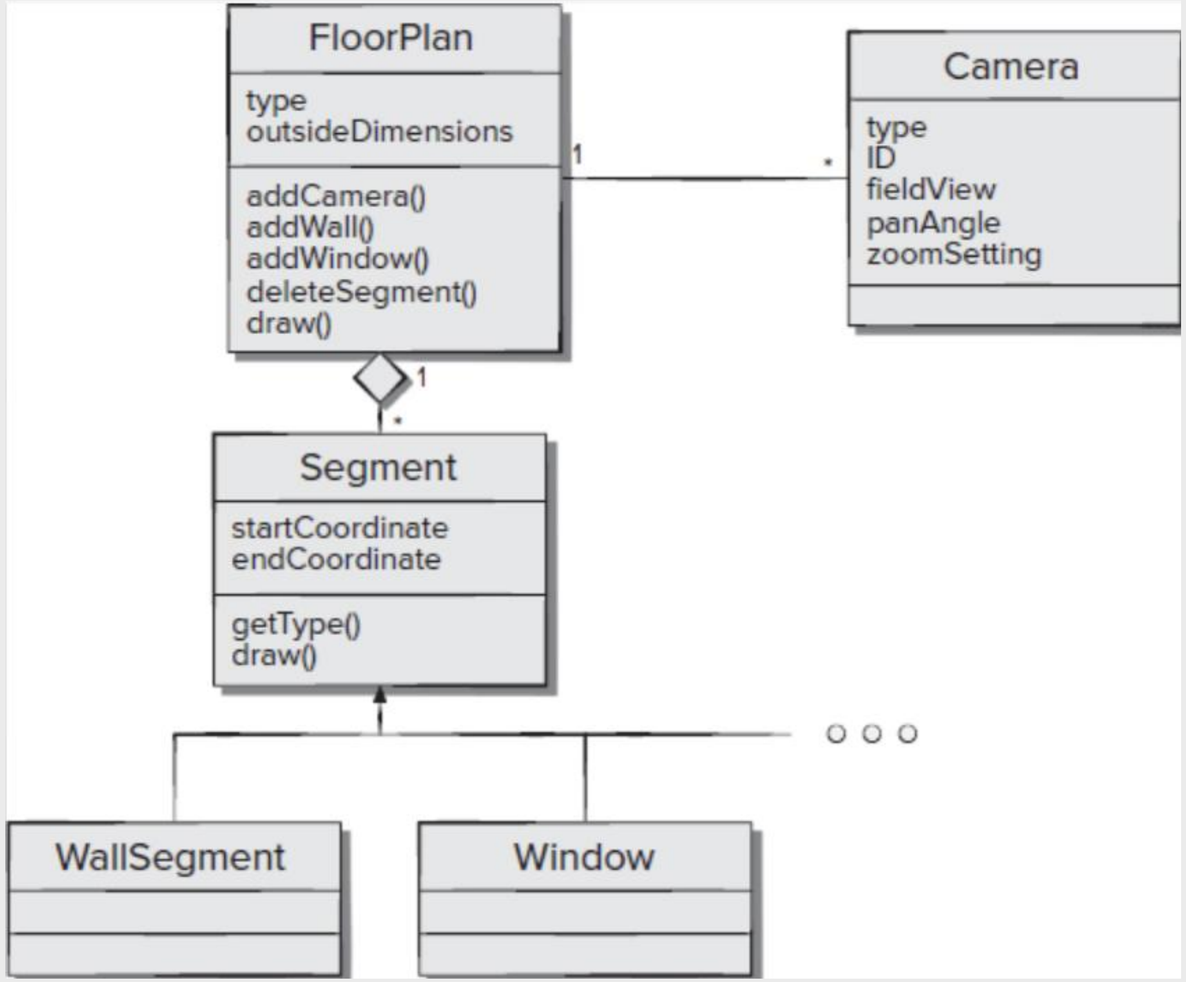
3. High Cohesion
- Methods and attributes focus on a **single responsibility**
- Class does one thing well

4. Low Coupling
- Minimal dependency on other classes
- Follows the **Law of Demeter**: talk only to “neighbor” classes

DESIGN CONCEPTS

DESIGN CLASSES

- Shows how an **analysis concept** becomes a **design-ready class**
- Demonstrates:
  - Encapsulation of data + behavior
  - Composition (FloorPlan → Segment)
  - Controlled collaboration with Camera



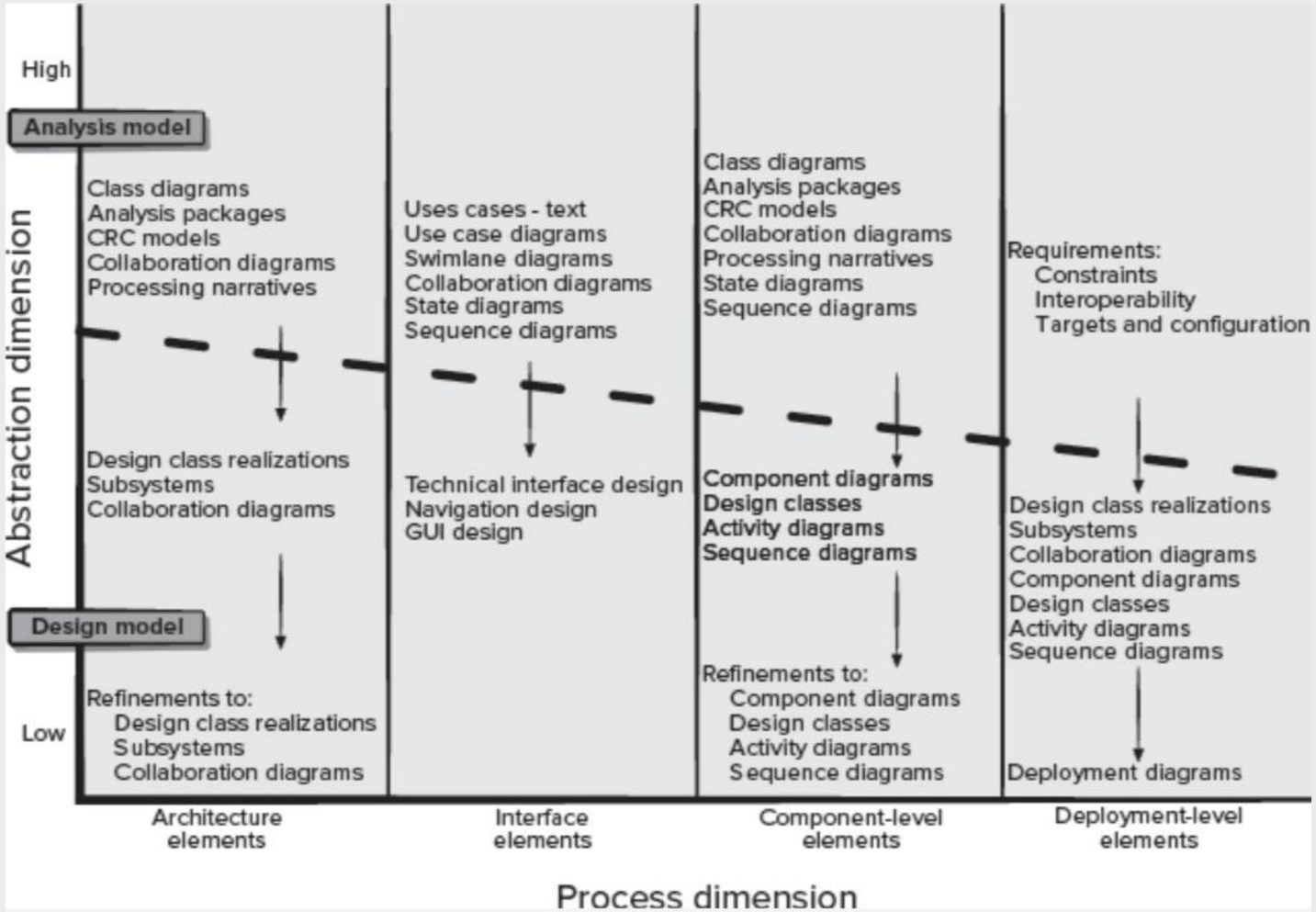
THE DESIGN MODEL

FROM ANALYSIS TO IMPLEMENTATION

What this diagram shows

- The **transition from analysis model → design model**
- How software design **progressively reduces abstraction**
- How different **design views evolve in parallel**

Design is **not a single step**, but a **systematic refinement** of analysis artifacts into implementable structures across architecture, interface, components, and deployment.



THE DESIGN MODEL

TWO KEY DIMENSIONS

1. Abstraction Dimension (Vertical)

Top (High abstraction)

- Focus: *what the system should do*
- Artifacts: use cases, analysis class diagrams, behavioral models

Bottom (Low abstraction)

- Focus: *how the system will be built*
- Artifacts: design classes, components, deployment diagrams

Design gradually moves from **conceptual** to **technical** detail

2. Process Dimension (Horizontal)

Design is refined across **four complementary views**:

Architecture Elements

- Subsystems and major structural decisions
- Defines the system's backbone

Interface Elements

- User interface and external interfaces
- Navigation, interaction, technical interfaces

Component-Level Elements

- Internal logic and component behavior
- Design classes, activity and sequence diagrams

Deployment-Level Elements

- Physical realization of the system
- Nodes, environments, and deployment constraints

THE DESIGN MODEL

DESIGN MODELING PRINCIPLES

Principle 1. Design should be traceable to the requirements model.
Principle 2. Always consider the architecture of the system to be built.
Principle 3. Design of data is as important as design of processing functions.
Principle 4. Interfaces (both internal and external) must be designed with care.
Principle 5. User interface design should be tuned to the needs of the end user.
Principle 6. Component-level design should be functionally independent.
Principle 7. Components should be loosely coupled to one another and to the external environment.
Principle 8. Design representations (models) should be easily understandable.
Principle 9. The design should be developed iteratively.
Principle 10. Creation of a design model does not preclude an agile approach.

People
Innovation
Excellence

REFERENCES

- Roger Pressman and Bruce Maxim. (2020). Software Engineering: A Practitioner's Approach. 9th. McGraw Hill. New York. ISBN: 978-1259872976.